

```

25%
/dev/shm:
0%
/tmp:
29%
/var/log:
8%

```

Salt State

Salt State is a component in SaltStack that holds the configuration data on the master, and which gets transferred to all minions, on demand.

The collection of state files makes up the 'State Tree'. The Salt file server must be set up first. So, edit the master configuration file `'/etc/salt/master'` and uncomment the following lines:

```

file_roots:
  base:
    - /srv/salt

```

The data will be stored in a simple serialisation format (YAML) in a master file server. YAML stands for 'YAML Ain't Markup Language'.

We can set a different folder for the base in the master and also define separate folders under the base, depending on the environments that we support. For instance, if you have the `dev`, `test` and `prod` environments, then the directory structure might look like what is shown below:

```

file_roots:
  base:
    - /srv/salt
  dev:
    - /srv/salt/dev
  test:
    - /srv/salt/test
  prod:
    - /srv/salt/prod

```

We could keep common Salt States like `user`, `group` and `sudo` in the base structure, and other configurations, which might be specific to the environments, in their respective folders.

A Salt State consists of identifiers, as well as state module and function declarations. It might also include function parameters.

```

nginx:          #Identifier
pkg:            #State Module
- installed     #State Function

nginx:          #Identifier

```

```

pkg.installed  #State Module and Function combined.

```

```

/etc/nginx:
file.directory: # State module and function
- user:nginx    #State function parameter

```

The top file

The top file contains the list of all the Salt States to be used. It maps the name with the SLS module that gets loaded on a certain minion via the state system. By default, it understands '*' as all minions. We can specify a set of minions or even a number of minions, using regular expressions or a grouping criterion, e.g., environment.

The file will be within the folder that we mark as base. We could also keep a top file within each environment and refer these top files within the base top file.

Let's prepare the SLS file to install the Nginx Web server. With the `pkg` module we can install packages using Salt. Create the `nginx.sls` file in the base folder as shown below.

```

/srv/salt/nginx.sls:

nginx:
  pkg:
    - installed

```

If the top file were to use the above Nginx Salt State, then it would look like what follows:

```

/srv/base/top.sls:

base:
  '*':
    - nginx

```

Running the Salt State

A Salt State can be run in two ways.

- Salt Master could instruct the targeted minion to only run the selected module. The module is then downloaded, compiled and executed on the minion. Once completed, the minion will revert with a summary of all actions taken and all changes made.

```

salt 'myminion' state.sls nginx (Flow: nginx)

```

- The master could instruct the targeted minion to run as `state.highstate`. When a minion executes a `highstate` call, it will download the top file and attempt to match the expressions within the top file. When it does match an expression, the modules listed for it will be downloaded, compiled and executed. Once completed, the minion will give a summary of all actions taken and all changes made.